Final Documentation: Buffer Manager and Page System

1. Introduction

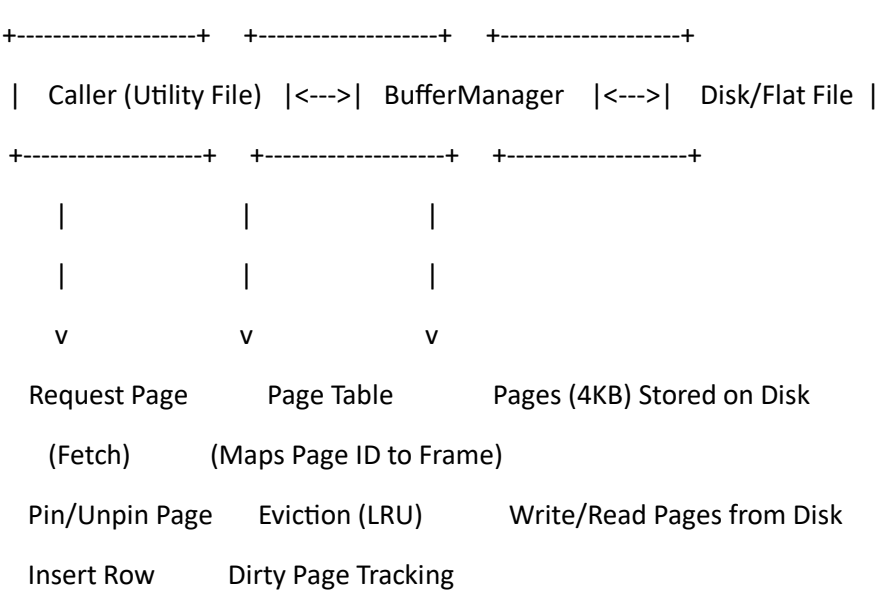
The Buffer Manager and Page System is a critical component designed to handle efficient storage and retrieval of movie data from a disk-based file system. This implementation provides a structured approach to managing data pages in memory while ensuring optimal performance. The system is composed of a Buffer Manager that oversees in-memory page management and an eviction policy that facilitates the replacement of pages when memory constraints are reached. Additionally, the Page class serves as the fundamental unit of storage, organizing data into fixed-size blocks to enable efficient reading and writing operations. This documentation provides an in-depth analysis of the design, implementation, and operational methodologies adopted for this system.

2. Design and Implementation

The design choices made for this system were driven by efficiency, simplicity, and scalability. The Buffer Manager was implemented using a HashMap to track the pages loaded into memory. Each page represents a 4KB block, which stores multiple rows of movie data, including a unique movie identifier and a title. The system is designed to handle frequent insertions and retrievals, making it particularly suited for workloads where data is predominantly appended rather than modified or deleted. The Buffer Manager ensures that pages are loaded into memory only when needed and employs an eviction policy based on the Least Recently Used (LRU) strategy. This policy determines which pages should be removed from memory when space is needed for new pages. Furthermore, the system incorporates a dirty page handling mechanism, ensuring that modified pages are written back to disk before eviction. To prevent active pages from being removed prematurely, a pinning mechanism is employed, allowing certain pages to be temporarily locked in memory.

Each page within the system is responsible for storing a fixed number of rows. The structure of each row consists of a movie ID, which occupies nine bytes, and a movie title, which spans 30 bytes. The total number of rows that a page can accommodate is determined by the page size and row size constraints, in our case, we use 4kb pages (4096 bytes). When a page reaches its capacity, a new page is created to continue storing additional rows. Serialization techniques are employed to ensure that data is efficiently stored on disk, allowing for quick retrieval and minimal overhead.

System Architecture:



3. Methodology and Functionalities

3.1 Buffer Manager Functionalities

The Buffer Manager is responsible for handling pages in main memory(Cache), which ensures efficient storage, retrieval from disk, and eviction from Buffer Pool after verifying dirty status(if page is modified). The following methods define its core functionalities:

- **getPage(int pageId):** This method is responsible for loading a page from buffer pool. If the requested page is not already in buffer pool, it is fetched from the disk storage and stored in the buffer pool. If buffer Pool is full while getting the page details, it calls evict method.
- **createPage():** This method is responsible to create a page object that stores page metadata and data content in the buffer pool. It will get added to the front of the queue and it is pinned and marked dirty as it is ready to insert the data.
- **evictPage():** This method evicts the page when buffer pool is full. It implements the LRU eviction policy. It identifies the least recently used page (unpinned page) and removes it from buffer pool to make space for new pages. Before eviction, any dirty pages are written back to disk to prevent data loss.
- **pinPage(int pageId):** This method pins a page ensures that it is protected from eviction. This is useful when a page is actively being used and should not be removed from memory.
- **unpinPage(int pageId):** This method unpins a page which signals that it can now be considered for eviction if necessary.
- **writePageToDisk(int pageId, Page page):** This method writes the specified page to disk at the appropriate location and It ensures that the data is stored in a binary format for efficiency. The records are stored in 9 byte chucks and the last byte is the amount of records stored on the page.
- **loadPageFromDisk(int pageId):** This method retrieves a page from disk, reconstructs its data, and returns a Page object containing the stored rows.
- **getNextPageId():** This method gives the id for next page when createPage() is called. The ids assigned to the next new page are negative, this to ensure there are no conflicting ids with pages that are already on the disk. The ids are resigned when these pages are written to the disk.
- **populateDisk(int numRecords, String filepath):** This method is used to populate the disk with data from our tsv file. It filled a page completely with records before write on to the next page. The text data is serialized into binary data before writing on the file.

3.2 Page Functionalities

The Page class defines the structure and behaviour of individual pages. Each page is responsible for storing rows of movie data and managing serialization and deserialization processes. The core methods include:

- **getRow(int rowId):** This methods get the row based on the slot id of the record.
- **insertRow(Row row):** This method inserts a new row into the page. If the page is full, the insertion is rejected, and a new page must be created. During insertion the row is marked dirty.
- **isFull():** This method checks whether the page has reached its capacity.
- **getPid():** This method returns the page id of the page.
- **reassignPageId(int pageId):** This method updates the current page id of the page. This method is used before the new created page is written to the disk.
- **incrementPinCount():** This method increments pin count which is stored in a property of page.
- **decrementPinCount():** This method decrements pin count which is stored in a property of page.
- **getPinCount():** This method returns pin count.
- **getAllRows():** Returns an array containing all the rows currently stored in the page.
- **getBytesToPad():** This method returns the number of bytes between the last record stored in the page, and the last byte of the page. This method is used so that the page can be filled with zeros between the last record, and the last byte, where the row count of the page is stored.
- **setAllRows(Row[] rows):** This method is exclusively called by loadPagefromDisk(). It sets the binary data loaded from disk in the page object.
- **setRowCount(int rowCount):):** This method is exclusively called by loadPagefromDisk(). It sets the row count in page object which is stored in the last byte of each page on the disk.
- **getRowCount():** This method return row count.
- **markDirty():** When a page is modified, it is marked as dirty. This ensures that changes are persisted to disk before the page is evicted from memory. It makes the Boolean value as true.
- **markNotDirty():** This method removes the dirty tag from the page which is it changes the boolean value to false.
- **getDirtyStatus():** This method gets page status in terms of dirty/not dirty.
- **deserializeRows():** This method reads serialized data from disk and reconstructs the page.
- **getDeserializedRows():** This method returns the deserialized rows

3.3 Utility Functions for Disk Management

The system provides additional utilities for managing disk storage, including:

- **loadDataset(BufferManager bf, String filepath**): Follows the pattern detailed in the lab description. It loads the buffer pool with pages from disk, while also testing the LRU eviction policy in conjunction with the pin functionality. It first creates a new page, attempts to get it from the buffer pool, inserts rows into the page, unpins it once leaving it still pinned, loads enough pages to evict a page while still being pinned, unpins it, evicts it from the buffer pool, gets it by loading it from disk, and finally inserts more rows into the retrieved page from disk.

4. Prerequisites:

1. Have JDK set up on your device. The instructions can be found here: https://www.jetbrains.com/help/idea/sdk.html#change-project-sdk

2. Have Gradle set up on your device. The instructions can be found here : https://www.jetbrains.com/help/idea/getting-started-with-gradle.html

   Make sure to add following dependencies in build.gradle file:

```
{
// JUnit 5 for testing
testImplementation 'org.junit.jupiter:junit-jupiter-api:5.9.2'
testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.9.2'
// Mockito for JUnit 5 integration
testImplementation 'org.mockito:mockito-core:4.11.0'
testImplementation 'org.mockito:mockito-junit-jupiter:4.11.0'
// This dependency is used by the application.
implementation 'com.google.guava:guava:32.1.2-jre' // Explicitly define Guava version
}
```

3. Download the dataset name : title.basics.tsv.gz from https://datasets.imdbws.com/
   a. Unzip the file and store it the path under: **app/src/main/java/project_645/DB files**
4. Use the current testdb.dat file which is place under **app/src/main/java/project_645/DB files**
5. Change the path name and filename in main file.
6. Have GitHub configuration on system to import the main repository mentioned in next section.

5. Execution

5.1 To set up and run the system, follow these steps:

1. Open Terminal
2. Clone the repository using the command:

   git clone https://github.com/swanky-rt/main.git
3. Navigate to the project directory: **cd main**
4. Make sure all the steps in Prerequisite section are completed.
5. Build the project **: ./gradlew build**
6. Run the project: **./gradlew run**
7. Make sure build is passed, if not please check the trouble shooting section for help.

5.2 Loading Data

To load the dataset into the system:

1. Ensure that the title.basics.tsv file is placed in the correct directory.
2. Execute the data loading script:
3. java -jar buffer_manager.jar

Some Important Note:

To maintain the offset of a page, we have kept the flow as in when a page with negative id(created but not written to disk) is written to the file. It is reassigned an ID which corresponds to the page at the file size, appending a new page of contents to the file.

In case of eviction, we are maintaining the sync in offset as we evict the page from buffer pool but needs to write to disk in case if it is dirty and also it needs to be stored at its offset in disk which is pageId *page size. To do maintain the flow without any bug, we are assigning negative id to a new page.

6. Conclusion

The Buffer Manager and Page System provides an efficient mechanism for handling large-scale movie data storage and retrieval. With its structured page-based storage, LRU-based eviction policy, and robust disk management utilities, the system is designed to balance memory constraints with performance efficiency. Although it currently operates under the assumption of an insert-only workload with no indexing support, future improvements could enhance its scalability and query optimization. This documentation serves as a comprehensive guide to the design, implementation, and usage of the system, ensuring clarity for both developers and users.

Output: After executing main class i.e utility file(load from dataset), following should be the output(for the reference):

Please note that a disk file is provided for you at the path specified in main. If you would like to create your own, do so, and update the filename in the Main class.

If you would like to populate the file with entries, which is necessary for “loadDataset” to run, do so by calling “bufferManager.populateDisk()” , which populates the disk with as many records as provided with the tsv file for movie info at the provided file path. This will store records continuously in a packed data format up until record n, meaning that all pages will be completely populated except for the last one if a non-multiple of 105 is passed, otherwise, all pages will be completely populated.

```
> Task :app:run
-1the page numberthe page 0 is pinned
the page 1 is pinned
the page 2 is pinned
the page 3 is pinned
the page 4 is pinned
the page 5 is pinned
the page 6 is pinned
the page 7 is pinned
the page 8 is pinned
the page 9 is pinned
the page 10 is pinned
the page 11 is pinned
the page 12 is pinned
the page 13 is pinned
the page 14 is pinned
```

UTs coverage:

To ensure the correctness and performance of the system, a rigorous testing strategy was employed. The testing methodology includes unit tests and edge case handling.

There are 3 files i.e. BufferImpl.java, PageImpl.java and Utility.java.

We have written the UTs to cover the above files and below is the summary of the coverage.

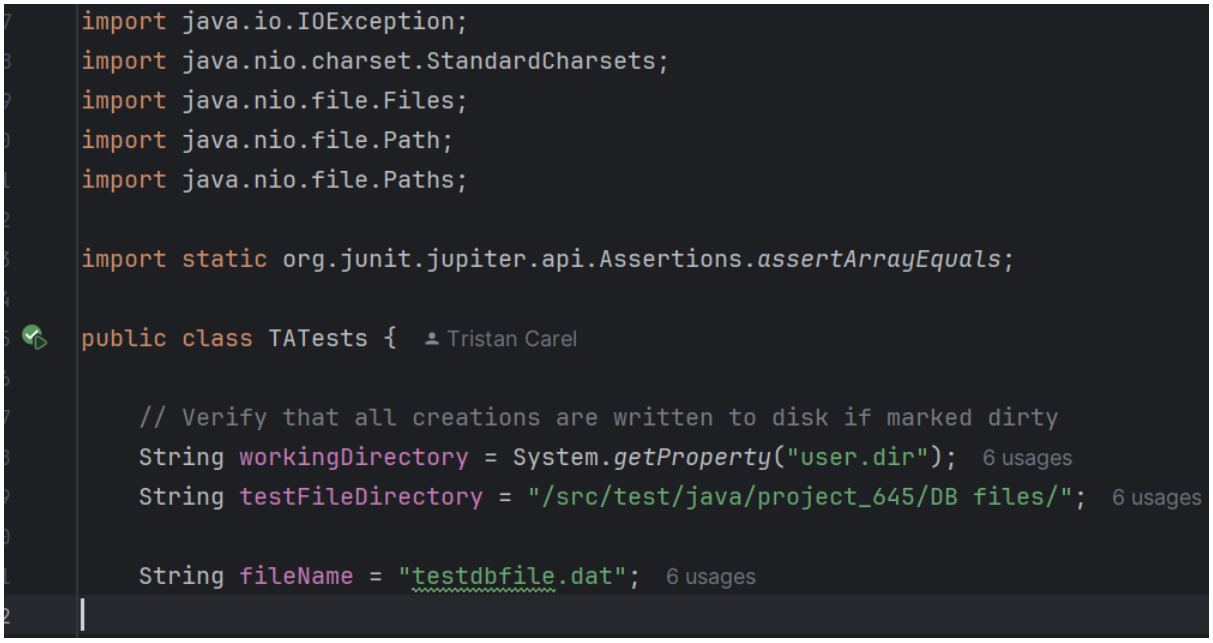
Element	Class, %	Method, %	Line, %	Branch, %
project_645	83% (5/6)	94% (35/37)	95% (208/218)	93% (67/72)
BufferManager	100% (1/1)	50% (1/2)	66% (2/3)	100% (0/0)
BufferManagerImpl	100% (1/1)	100% (10/10)	100% (125/125)	92% (46/50)
Main	0% (0/1)	0% (0/1)	0% (0/9)	100% (0/0)
PageImpl	100% (1/1)	100% (19/19)	100% (37/37)	91% (11/12)
Row	100% (1/1)	100% (3/3)	100% (7/7)	100% (0/0)
Utilities	100% (1/1)	100% (2/2)	100% (37/37)	100% (10/10)

Important link:

Below is the Github link for the folder for the BufferPool implementation(main fodler) and UTs(test folder)

https://github.com/swanky-rt/main/tree/main/app/src

We have executed TATests.java(UTs provided by Tas) and it passed for the logic that we have implemented.



Trouble Shooting:

- Build failure:
  - Check the error messages by running : **./gradlew build –stacktrace**
  - In case of compilation error check the syntax and imports
  - Check for missing dependencies in build.gradle
  - Run **./gradlew clean** and try again.

- Mockito error:
  a. Add these dependencies in your build.gradle:

     dependencies { testImplementation 'org.mockito:mockito-core:5.3.1' testImplementation 'org.mockito:mockito-junit-jupiter:5.3.1' }

  b. Make sure **import static org.mockito.Mockito.*;** is imported in your test files.

- Gradle error:

  a. Check Gradle version by running: **gradle -v**
  b. If error is ‘Could not resolve dependencies’ : Run **./gradlew dependencies**

c. If error regarding Plugin version issues. Update gradle by running:
./gradlew wrapper --gradle-version 8.0 --distribution-type all

d. If any gradle error pertains try rebuilding everything by :

1. **rm -rf .gradle/ build/**
2. **2 ./gradlew build**

- Path error:
  a. File Not Found (NoSuchFileException)
     i. Ensure the file exists at the expected path.
     ii. If using relative paths, switch to absolute paths.
     iii. Try creating the missing file manually.
  b. Make sure your directories are correct:
     i. mkdir -p "C_your_folder_path\main\app\src\main\java\project_645\DB files"
     ii. touch "C:\ your_folder_path\main\app\src\main\java\project_645\DB files/testdb.dat"

7**. Contribution Breakdown**

- **Aarti Kumari**: Buffer Manager implementation, Architecture design, UTs and Documentation over view.
- **Tristan Carel**: Page class and file I/O implementation and documentation flow.
- **Rithika Florian Johnson**: Unit Testing and Gradle configuration for the project.
- **Shreyaa Dani**: Unit Testing and documentation implementation.